

Angular Avançado · Módulos 2, 5, 7 e 8

Sessão 14

RxJS na store, performance e fecho do curso

João Gonçalves · joaogoncalves.net · Making the complicated simple

Onde ficámos na Sessão 13

As regras verificadas e uma store com forma

Sheriff

- `sheriff.config.ts`: tags + `depRules`
- `npm run lint:arch` → No issues found
- Cada exceção é uma decisão escrita

NgRx Signal Store

```
export const FlightStore = signalStore(
  { providedIn: 'root' },
  withState(initialState),
  withProps(…),         // o resource
  withMethods(…),       // patchState
  withComputed(…),      // route, selectedCount
);
```

Hoje: a store passa a falar **RxJS**, a app fica mais **rápida** — e fechamos o curso.

Confirmar: `git pull origin main` (tag `sessao-13`) · `npm start` · Login.

Agenda de hoje

01	**Signals e RxJS na store**	toSignal · toObservable · rxMethod · withHooks	≈ 38 min
02	**Eventos entre stores**	A Event API num slide	≈ 7 min
03	**Zoneless e change detection**	Quem avisa o Angular? · OnPush · Profiler	≈ 17 min
04	**@defer**	Carregar mais tarde · triggers · prefetch	≈ 17 min
05	**SSR e hydration**	Uma visão geral	≈ 10 min
06	**Fecho do curso**	O percurso · o Skyboard · para continuar	≈ 12 min

A escuro: os blocos com código. Os outros são conversa com slides.



Bloco A · Signals e RxJS

Signals e RxJS na store

Um pedido por pesquisa, nem um a mais

Signals ou RxJS?

Não é um ou outro: cada um tem o seu trabalho

Signals: o estado

- Um **valor atual**, sempre disponível
- Ler é chamar: `from()`
- Síncrono, sem `subscribe`
- `computed` deriva, o template lê
- O que **é**: o filtro, a lista, o cesto

RxJS: o tempo

- Uma **sequência de eventos** ao longo do tempo
- Operadores: `debounceTime`, `switchMap`, `retry`…
- Cancelar o pedido anterior, esperar, juntar streams
- Precisa de `subscribe` (e de o desligar)
- O que **acontece**: teclas, pedidos, timers

Signals para o **estado**, RxJS para o **tempo**. A Sessão 3 começou pelos signals; hoje juntamos os dois.

A ponte: toSignal e toObservable

@angular/core/rxjs-interop

```
// Observable → Signal
readonly now = toSignal(
  interval(1000).pipe(map(() => new Date().toLocaleTimeString())),
  { initialValue: '' },
);
```

```
// Signal → Observable
readonly from$ = toObservable(this.store.from).pipe(debounceTime(300));
```

toSignal

- Faz o `subscribe` e o `unsubscribe` sozinho
- Sem `initialValue` → começa `undefined`

As duas

- Só num **injection context** (construtor, campo)
- Hoje: o `rxMethod` faz esta ponte por nós

rxMethod: um método que aceita um stream

@ngrx/signals/rxjs-interop

Entrada

valor · `signal` · Observable

pipe(…)

`debounceTime` · `switchMap` · …

tap → patchState

o resultado vai para o estado

```
_load: rxMethod<string>(
  pipe(debounceTime(300), switchMap((name) => …), …)
),

store._load(store.name);   // um signal: cada valor novo entra no
pipe
```

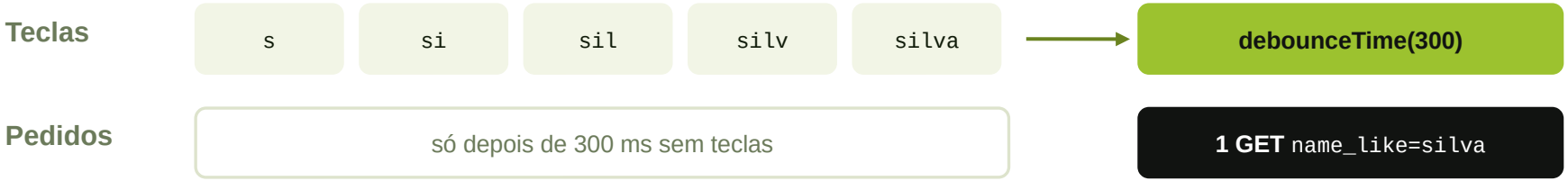
Sem trabalho manual

- Sem `subscribe`
- Desliga-se com o injector de quem o chamou

Por dentro é um `effect`: o signal muda → o valor entra no stream.

Escrever "silva": quantos pedidos?

debounceTime, distinctUntilChanged, switchMap — e onde fica o catchError



switchMap

- Novo nome com um pedido a meio → **cancela** o anterior
- A resposta antiga nunca chega a escrever no estado
- `distinctUntilChanged`: o mesmo nome não repete o pedido

```
O catchError fica DENTRO
switchMap((name) =>
  client.find(name).pipe(
    tap(…),
    catchError(() => { …; return EMPTY; }),
  ),
)
```

Fora do `switchMap`, o primeiro erro terminava o stream: a pesquisa morria para sempre.

withHooks: o ciclo de vida da store

onInit e onDestroy

```
withHooks((store) => ({
  onInit(): void {
    store._load(store.name);
  },
  onDestroy(): void { … },
})),
```

Quando correm?

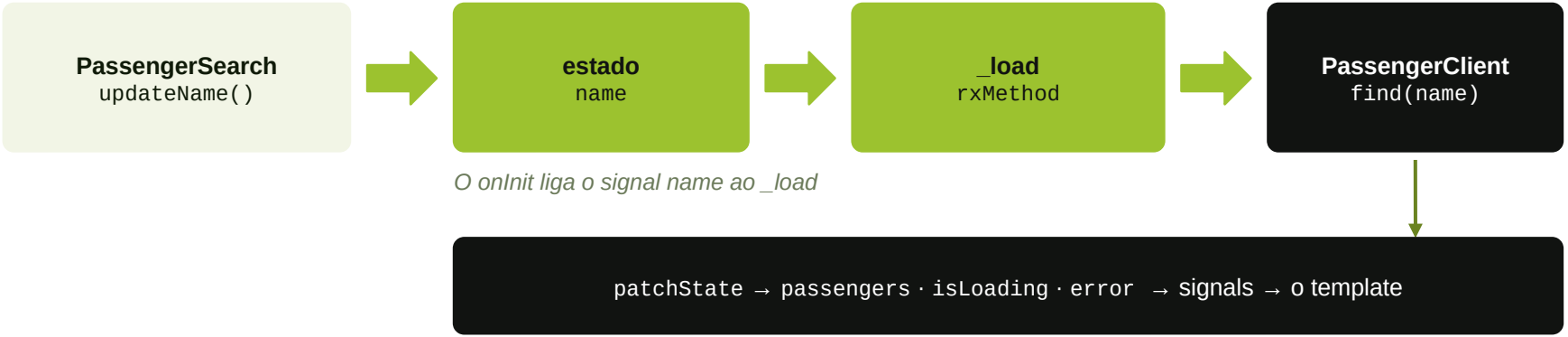
- `onInit`: quando a store é criada — no primeiro `inject(PassengerStore)`
- `onDestroy`: quando o injector dela morre
- Store `root`: o `onDestroy` só no fim da app

Porquê aqui e não no componente?

A store liga-se a si própria: o componente só muda o `name`; **quem decide carregar é a store**. Num injection context — por isso o `rxMethod` pode receber um signal.

O que vamos construir: o PassengerSearch

Hoje a página "Passengers" deixa de dizer Coming soon



Snippet	Ficheiro	Camada
R1	db.json · config-service.ts · data/passenger-client.ts	data (usa util)
R2	feature-booking/passenger-search/passenger-store.ts	feature
R3	passenger-search.ts · passenger-search.html	feature



Snippet R1 · da teoria ao código

Agora, vamos escrever o código

- No `db.json`: uma coleção `passengers`
- No `ConfigService`: o `passengersUrl`
- Um `PassengerClient` em `data`, com `find(name) → Observable`

db.json · shared/util-common/config-service.ts · ticketing/data/passenger-client.ts

R1 · Os passageiros na API

Um client como os outros — só que devolve um Observable

```
// db.json — no fim
"passengers": [
  { "id": 1, "name": "Ana Silva",
    "boarded": false },
  …
]

// config-service.ts
passengersUrl = '…:3000/passengers'
```

data/passenger-client.ts

```
@Service()
export class PassengerClient {
  private readonly config = inject(ConfigService);
  private readonly http = inject(HttpClient);

  find(name: string): Observable<Passenger[]> {
    return this.http.get<Passenger[]>(
      this.config.passengersUrl,
      { params: { name_like: name } });
  }
}
```

Experimentar

`localhost:3000/passengers?name_like=silva` → Ana Silva e Inês Silva. Com `name_like=` vazio → todos.



Snippet R2 · da teoria ao código

Agora, vamos escrever o código

- `withState`: `name`, `passengers`, `isLoading`, `error`
- `updateName`: só muda o estado (`patchState`)
- `_load`: um `rxMethod` com `debounceTime` e `switchMap`
- `withHooks`: no `onInit`, ligar o signal `name` ao `_load`

ticketing/feature-booking/passenger-search/passenger-store.ts

R2 · A PassengerStore: estado e intenção

A mesma forma da FlightStore

```
type PassengerSearchState = {
  name: string;
  passengers: Passenger[];
  isLoading: boolean;
  error: string | null;
};

export const PassengerStore = signalStore(
  { providedIn: 'root' },
  withState(initialState),
  withProps(() => ({ _passengerClient: inject(PassengerClient) })),
  withMethods((store) => ({
    updateName(name: string): void {
      patchState(store, { name });
    },
    _load: rxMethod<string>( … ),   // slide seguinte
  })),
```

updateName

- Não faz pedidos
- Só guarda a **intenção**

initialState

- `name: ''` → no início, **todos**
- `isLoading`, `error`: geridos pela store

R2 · rxMethod e withHooks

O estado muda → o stream reage → o estado muda

```
_load: rxMethod<string>(
  pipe(
    debounceTime(300),
    distinctUntilChanged(),
    tap(() => patchState(store, { isLoading: true, error: null })),
    switchMap((name) =>
      store._passengerClient.find(name).pipe(
        tap((passengers) => patchState(store, { passengers, isLoading: false })),
        catchError(() => { patchState(store, { error: …, isLoading: false });
                           return EMPTY; }),
      ),
    ),
  ),
),

withHooks((store) => ({ onInit(): void { store._load(store.name); } })),
```

`store.name` **sem** `()`: passa o signal, não o valor — cada nome novo entra no `pipe`.



Snippet R3 · da teoria ao código

Agora, vamos escrever o código

- `linkedSignal` com `set → updateName` (como no `FlightSearch`)
- Um `form` com um campo; o template lê `passengers`, `isLoading`, `error`
- Experimentar com a consola: escrever "silva" depressa

passenger-search.ts · passenger-search.html

R3 · O PassengerSearch

O componente não sabe que existe RxJS

```ts
export class PassengerSearch {
  private readonly store = inject(PassengerStore);

  protected readonly name = linkedSignal(
    () => this.store.name(),
    { set: (value) => this.store.updateName(value) });
  protected readonly nameForm = form(this.name);

  protected readonly passengers = this.store.passengers;
  protected readonly isLoading = this.store.isLoading;
  protected readonly error = this.store.error;
}
```

```
passenger-search.html
<input [formField]="nameForm" />

@for (p of passengers();
      track p.id) {
  <li>{{ p.name }}</li>
} @empty { … }
```

Entrar: **1 GET** (o `onInit`)

"silva" depressa: **1 GET**

Flights e voltar: **0 GET**

Resource ou rxMethod?

Duas ferramentas para carregar dados numa store

	Resource (FlightStore)	rxMethod (PassengerStore)
Pedido	Declarativo: depende de signals	Um pipe de operadores RxJS
Cancelar o anterior	Sim, sozinho	Com switchMap (ou exhaustMap, concatMap…)
Esperar (debounce)	No form: debounce()	Na store: debounceTime()
isLoading, error	Vêm de graça	Escritos à mão no estado
Gravar (POST, PUT…)	Não: os resources são para ler	Sim: exhaustMap, concatMap
Quando	Ler dados que dependem de filtros	Tempo, retry, polling, streams a juntar
Exemplos reais	Pesquisa com filtros, detalhe de um voo	Guardar duas vezes, polling, WebSocket

Para ler dados, o **resource** chega. O `rxMethod` ganha quando o problema é **tempo**.

Paragem segura: a app compila e funciona.



Bloco B · Eventos

Eventos entre stores

A terceira saída da Sessão 13 — só slides

Stores a falar com stores: três saídas

Recap da Sessão 13 — hoje a terceira

1 · Ler de baixo

Uma store lê signals de uma camada **abaixo**

`FlightStore` → `AuthStore` (util)

2 · Facade

Um serviço acima **orquestra** as stores

`CheckInFacade` → `FlightStore` + `LuggageStore`

3 · Eventos

Quem faz a ação **anuncia**; quem quer, reage

`passengerBoarded` → 1, 2 ou 10 stores

As dependências descem, a coordenação fica numa facade, o que atravessa fronteiras viaja como **evento**.

A Event API num slide

@ngrx/signals/events — não fazemos no Skyboard

```ts
// 1 · o que aconteceu
export const boardingEvents = eventGroup({
  source: 'Boarding List',
  events: { passengerBoarded: type<{ passengerId: number }>() },
});

// 2 · uma store que reage (podem ser várias)
withReducer(
  on(boardingEvents.passengerBoarded, (_event, state) => ({
    boardedCount: state.boardedCount + 1,
  })),
),

// 3 · quem emite não conhece quem ouve
private readonly dispatch = injectDispatch(boardingEvents);
this.dispatch.passengerBoarded({ passengerId: 4 });
```

Ganha-se

- Desacoplamento máximo
- Mais um interessado = zero alterações em quem emite

Paga-se

- Fluxo indireto: quem reage a isto?
- Mais ficheiros, mais nomes



Bloco C · Zoneless

Zoneless e change detection

Quem avisa o Angular de que algo mudou?

Quem avisa o Angular?

Change detection: atualizar o DOM quando os dados mudam

Antes: zone.js

- Interceta **tudo**: `setTimeout`, eventos, XHR
- Depois de cada um: verifica a app inteira
- Funciona "por magia" — e paga-se em tempo
- Um polyfill a mais e stack traces confusos

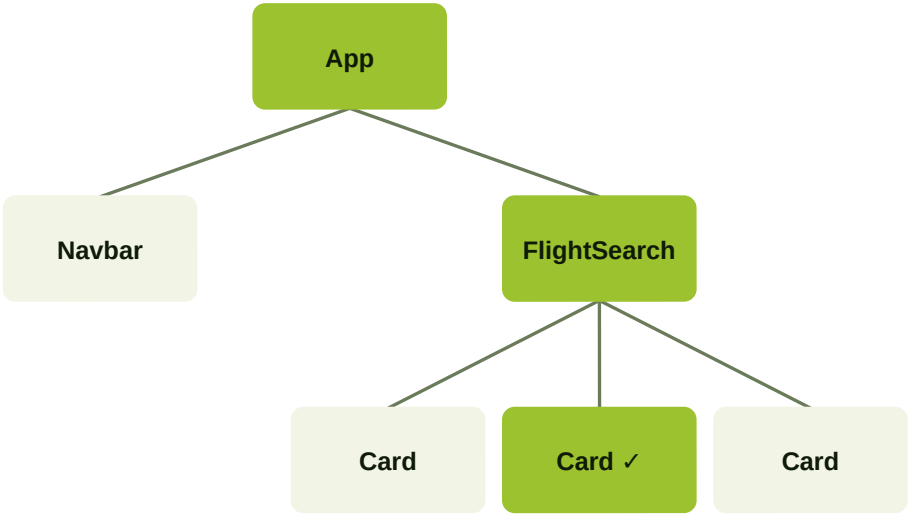
Agora: zoneless

- Sem zone.js — projetos novos já são assim
- O Angular só atualiza quando é **avisado**:
  - um **signal** lido no template muda
  - um **evento** do template (`(click)`…)
  - `AsyncPipe` · `markForCheck()`

O Skyboard é zoneless desde a Sessão 1: o `angular.json` não tem `polyfills` com `zone.js`.

OnPush: só o que mudou

O default no Angular 22



Marcar um voo no cesto: só o caminho até ao card que mudou

OnPush + signals

- Um componente só é verificado se foi **marcado**
- Signal lido no template mudou → marca esse componente
- Os outros ramos ficam como estão
- Exige **imutabilidade**: um objeto novo, não mutar (o spread do cesto)

Desde o Angular 22 o CLI nem escreve `changeDetection: OnPush`: é o default.



Snippets Z1–Z2 · da teoria ao código

Agora, vamos escrever o código

- No `About`: uma hora atualizada com `setInterval` — num campo normal
- Ver o que acontece (e o que **não** acontece)
- Corrigir com um `signal` e um `DestroyRef`

shell/about/about.ts · about.html

Z1 · Um relógio que não anda

Um campo normal mudado dentro de um setInterval

```
protected now = new Date().toLocaleTimeString();

constructor() {
  setInterval(() => {
    this.now = new Date().toLocaleTimeString();
    console.log('tick', this.now);
  }, 1000);
}

<!-- about.html -->
<p>Hora: {{ now }}</p>
```

O que se vê

- A hora **parada**
- A consola: `tick` a cada segundo

Dois problemas

- Ninguém avisa o Angular
- Ir para Home: os `tick` **continuam** — um leak

Com zone.js funcionava: a zone intercetava o `setInterval` e verificava tudo, a cada segundo.

Z2 · Um signal e um DestroyRef

O aviso e a limpeza

```
protected readonly now = signal(new Date().toLocaleTimeString());

constructor() {
  const id = setInterval(() => {
    this.now.set(new Date().toLocaleTimeString());
  }, 1000);

  inject(DestroyRef).onDestroy(() => clearInterval(id));
}

<p>Hora: {{ now() }}</p>
```

set()

- É o aviso: o template lê `now()`

DestroyRef

- Sair do About → `clearInterval`
- Os `tick` param

Em RxJS: `toSignal(interval(1000).pipe(map(…)), { initialValue: '' })` faz as duas coisas (slide 6).

O Profiler das Angular DevTools

Ver a change detection a trabalhar — só demo

1. DevTools → **Angular** → **Profiler** → Start recording
2. Em Flights: escrever no From, marcar um voo no cesto
3. Stop → cada barra é um ciclo de change detection
4. Clicar numa barra → que componentes foram verificados e quanto demoraram

Marcar um voo: aparecem o `FlightSearch` e **um** `FlightCard` — não a app inteira.



Bloco D · @defer

@defer: carregar mais tarde

O que não é preciso já, não pesa no arranque

@defer: um pedaço do template num chunk à parte

Deferrable views

```
@defer (on viewport) {
  <app-recommendations />
} @placeholder (minimum 300ms) {
  <app-skeleton />
} @loading (after 150ms; minimum 300ms) {
  <app-spinner />
} @error {
  <p>Could not load.</p>
}
```

`@placeholder`	antes do trigger
`@loading`	enquanto o chunk descarrega
`@error`	se o download falhar
`@defer`	o conteúdo, quando chega

Só os componentes usados **apenas** dentro do `@defer` vão para o chunk à parte.

Triggers e prefetch

Quando carregar — e quando descarregar

Trigger	Quando
`on idle`	O browser está livre (o default)
`on viewport`	O placeholder entra no ecrã
`on interaction`	Clique ou tecla no placeholder
`on hover`	O rato passa por cima
`on immediate · on timer(2s)`	Logo · depois de um tempo
`when (cond())`	Uma condição fica true

```
@defer (on interaction;
        prefetch on idle) {
  …
}
```

prefetch

- Descarrega cedo, **mostra** no trigger
- O clique já não espera pela rede

Não adiar o essencial: navegação e ações principais ficam fora do `@defer`.



Snippets D1–D3 · da teoria ao código

Agora, vamos escrever o código

- No `FlightSearch`: o `SimpleDelayStepper` dentro de um `@defer (on interaction)`
- Um `@loading` e um `prefetch on idle`
- `npm run build`: ver o chunk novo

feature-booking/flight-search/flight-search.html

D1 · Adiar o stepper

Um botão até ser preciso

flight-search.html

```
<!-- antes -->
<app-simple-delay-stepper [(value)]="maxDelay" />

<!-- depois -->
@defer (on interaction) {
  <app-simple-delay-stepper [(value)]="maxDelay" />
} @placeholder {
  <button type="button">Ajustar atraso</button>
}
```

Experimentar

- Network → filtro **JS**
- Abrir Flights: só o botão
- Clicar: **um .js novo** e o stepper aparece
- `+15 Minutes` funciona como antes

O `SimpleDelayStepper` continua nos `imports` — o compilador vê que só é usado no `@defer`.

D2 · @loading e prefetch

Mostrar quando é preciso, descarregar antes

```
@defer (on interaction; prefetch on idle) {
  <app-simple-delay-stepper [(value)]="maxDelay" />
} @placeholder {
  <button type="button">Ajustar atraso</button>
} @loading (minimum 500ms) {
  <small>A carregar…</small>
}
```

Ver o @loading

- Tirar o `prefetch` por um momento
- Network → **Slow 4G**
- Clicar → "A carregar…" ≥ 500 ms
- Repor tudo

prefetch on idle

O chunk chega **antes** do clique: o `@loading` quase nunca aparece.

minimum 500ms

Se aparecer, fica pelo menos meio segundo: sem "pisca".

D3 · O chunk no build

npm run build

```
Initial chunk files | Names                |  Raw size
main-NFDPIMIK.js    | main                 | 313.67 kB
styles-GAVA6QO6.css | styles               |  27.14 kB

Lazy chunk files    | Names                |  Raw size
chunk-CJwtj4X2.js   | ticketing-routes     |  83.06 kB
chunk-DxqTkSkb.js   | about                | 833 bytes
chunk-BlXl2E9v.js   | simple-delay-stepper | 774 bytes
```

`ticketing-routes`
loadChildren · Sessão 6

`about`
loadComponent · Sessão 7

`simple-delay-stepper`
@defer · hoje

774 bytes não mudam nada — o caso real são gráficos, editores e mapas com centenas de kB.



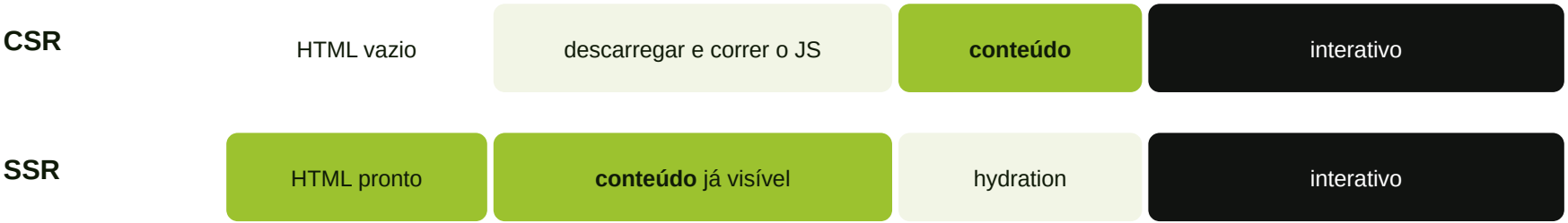
Bloco E · SSR

SSR e hydration

Uma visão geral — sem instalar no Skyboard

CSR e SSR: o que o utilizador vê

Client-side rendering vs server-side rendering



SSR ajuda

- Primeiro ecrã mais cedo (sites públicos, lojas)
- SEO e pré-visualizações de links

SSR custa

- Um servidor Node · código que corre nos dois lados
- Numa app interna atrás de login, raramente compensa

```
ng add @angular/ssr · ng new --ssr
```

Hydration, event replay e incremental hydration

Do HTML do servidor a uma app viva

Hydration

- O Angular **reaproveita** o DOM do servidor
- Liga eventos e estado, sem redesenhar

Event replay

- Cliques antes de a app estar pronta
- São gravados e **repetidos** depois

Incremental

- Cada `@defer` acorda **quando é preciso**
- Ligada por omissão no Angular 22

```
// app.config.ts
provideClientHydration(withEventReplay())

@defer (on viewport; hydrate on viewport) { <app-car-pane /> }
```

Render modes e cuidados

Hybrid routing: uma decisão por rota

```ts
// app.routes.server.ts
export const serverRoutes: ServerRoute[] = [
  { path: 'about', renderMode: RenderMode.Prerender },
  { path: 'ticketing/boarding', renderMode: RenderMode.Client },
  { path: '**', renderMode: RenderMode.Server },
];
```

Modo	O HTML é feito…
Prerender	no build (estático)
Server	no servidor, a cada pedido
Client	no browser (SPA clássica)

No servidor não há browser

- `window`, `document`, `navigator`, `localStorage` não existem
- Implementações diferentes por plataforma via DI — como o `provideLanguageService('browser')` do Skyboard
- Ou `isPlatformBrowser(inject(PLATFORM_ID))` para um caso pequeno

Fecho

Fecho do curso

14 sessões, 8 módulos, uma app

Resumo da sessão

RxJS na store

- `rxMethod`: valor, signal ou stream
- `switchMap` + `catchError` dentro
- `withHooks`: `onInit`

Eventos

- Quem emite não conhece quem ouve
- `eventGroup` · `withReducer`

Zoneless

- O Angular é **avisado**
- Signals + OnPush
- `DestroyRef` para limpar

@defer e SSR

- Chunk à parte, trigger, `prefetch`
- SSR: o primeiro ecrã mais cedo

Signals para o estado, RxJS para o tempo — e o Angular só faz o trabalho que lhe pedem.

O percurso: 8 módulos em 14 sessões

O programa aprovado, sessão a sessão

Módulo	Sessões	No Skyboard
1 · Arquitetura moderna	1 · 12 · 13	Standalone, domains e layers, Sheriff
2 · Control flow e rendering	1–3 · 14	@if, @for, @switch, @let, @defer
3 · Routing e DI	6–10	Lazy loading, providers e scopes, guards
4 · Componentes modernos	1–3	input, model, ng-content, FlightCard
5 · Signals e RxJS	1–5 · 14	computed, effect, linkedSignal, rxMethod
6 · HTTP moderno	2 · 4 · 9–10	httpResource, resources, interceptors
7 · Estado e formulários	9–11 · 13–14	Stores, Signal Forms, NgRx Signal Store
8 · Zoneless e performance	14	Zoneless, OnPush, @defer, SSR

O Skyboard no fim

A mesma app desde a Sessão 1 — git checkout sessao-01 … sessao-14

```
src/app/
├── shell/          navbar · sidebar · home · about
└── domains/
    ├── ticketing/
    │   ├── feature-booking/
    │   │     flight-search    (NgRx Signal Store)
    │   │     flight-edit      (Signal Forms)
    │   │     passenger-search (rxMethod)
    │   ├── feature-boarding/  boarding-list
    │   ├── ui/    flight-card · passenger-row
    │   └── data/  clients · tipos · schema
    └── shared/
        ui-common · ui-forms · util-auth · util-common
```

Por baixo

- Zoneless e OnPush
- Lazy routes + `@defer`
- Guards e interceptors funcionais
- `npm run lint:arch` limpo

Para continuar

O que ficou de fora — e por onde pegar

Temas para aprofundar

- Testes com Vitest (o CLI já o traz)
- Signal Forms a fundo: custom controls, validação async, Zod
- NgRx: entities, Event API, NgRx Toolkit (DevTools, mutations)
- SSR a sério: hydration, prerender, render modes
- Monorepos (Nx) e micro frontends (Native Federation)
- i18n

Onde

- **angular.dev** — guias e referência
- **ngrx.io/guide/signals** — Signal Store
- **github.com/softarc-consulting/sheriff**
- **Angular DevTools** — a extensão do browser
- **github.com/JoaoGoncalves/skyboard** — o código, sessão a sessão

A melhor forma de não esquecer: pegar no Skyboard e acrescentar uma feature sozinhos.



Fim do curso

Obrigado!

- Perguntas — sobre o curso ou sobre os vossos projetos
- O questionário de avaliação da MoOngy
- O Skyboard fica no GitHub: `sessao-01` … `sessao-14`

Extra · signalMethod

Como o rxMethod, mas sem RxJS

```
withMethods((store) => ({
  connectFlightId: signalMethod<number>((id) => {
    patchState(store, { flightId: id });
  }),
})),

// no componente: o id vem do router (withComponentInputBinding)
readonly id = input.required({ transform: numberAttribute });
constructor() { this.store.connectFlightId(this.id); }
```

Quando

Aceita um valor ou um signal e corre uma função a cada valor novo. **Sem** operadores: nada de debounce nem de cancelar pedidos — para isso, `rxMethod`.

Extra · Eventos com pedidos: withEventHandlers

O equivalente aos effects do NgRx clássico

```
withReducer(
  on(passengerEvents.searchTriggered, () => ({ isLoading: true, error: null })),
  on(passengerEvents.searchSucceeded, ({ payload }) =>
    ({ passengers: payload.passengers, isLoading: false })),
),
withProps(() => ({ _client: inject(PassengerClient), _events: inject(Events) })),
withEventHandlers((store) => ({
  search$: store._events.on(passengerEvents.searchTriggered).pipe(
    switchMap(({ payload }) => store._client.find(payload.name).pipe(
      map((passengers) => passengerEvents.searchSucceeded({ passengers })),
      catchError(() => of(passengerEvents.searchFailed({ error: '…' })))),
    )),
  ),
})),
```

Os reducers são síncronos e só mudam estado; os event handlers fazem o pedido e respondem com **outro evento**.